Airport (Malir) Campus

Final Year Project Report

BS (COMPUTER SCIENCE)

For

AI-Powered Candidate Recommendation System for smart Hiring

Submitted by

MAHAM JAVED	**(**IU05-0322-0103**)**
ALISHBA ALI	**(**IU05-0322-0082**)**
RABIA ASLAM	**(**IU05-0222-0378**)**

Supervisor

Dr. Affan Alim

Coordinator

Dr. Muhammad Naveed

DECLARATION

We hereby declare that this software, neither as a whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and the accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be the reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning

Maham Javed

Alishba Ali

Rabia Aslam

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) **"**AI-Based Recommendation System for Smart Hiring**"** was developed by *Rabia Aslam (IU05-0222-0378), Maham Javed IU05-0322-0108), Alishba Ali (IU05-0322-0082))* under the supervision of *"Dr. Affan"* and that in (his) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Dr.Affan Alim
(Supervisor)

Dr.Muhammad Naveed
(Coordinator)

Dr.Affan Alim
(HOD -CS)

Abstract

Xenon is a web-based intelligent recruitment system designed to support HR professionals and hiring managers in efficiently screening and ranking job candidates. Traditional recruitment processes are often time-consuming, subjective, and difficult to scale when faced with large volumes of applications. Xenon addresses these challenges by leveraging machine learning and natural language processing (NLP) techniques to automate candidate–job matching while maintaining transparency and usability. The system allows recruiters to upload job descriptions and candidate resumes, which are then analyzed using trained deep learning models to extract relevant skills, experience, and contextual information. Based on semantic similarity, the system generates match scores, ranked candidate lists, and explanatory reports that justify each recommendation. This approach reduces manual effort, improves consistency, and supports data-driven hiring decisions. Xenon is designed as an accessible, browser-based platform that emphasizes ease of use, real-time feedback, and explainable results. By integrating intelligent automation with a user-centered interface, the system enhances recruitment efficiency, minimizes bias, and improves the overall quality of hiring outcomes

Acknowledgement

All praise is due to Almighty Allah for granting us the strength, knowledge, and perseverance to successfully complete this project. We express our sincere gratitude to our project supervisor, **Dr. Affan Alim**, for his continuous guidance, valuable feedback, and technical insight throughout the development of this project. We are also thankful to **Dr. Muhammad Naveed**, Project Coordinator, for his support and encouragement. We extend our heartfelt appreciation to our families and friends for their constant motivation and patience. Their support played a vital role in the completion of this work.

Abbreviations

TTS	Text-to-Speech
UI	User Interface
ML	Machine Learning
.tflite	TensorFlow Lite
IDE	Integrated Development Environment
CNN	Convolutional Neural Network
MNIST	Modified National Institute of Standards and Technology
PE	Performance
US	Usability
RE	Readability
MA	Maintainability
PRE	Pre-Condition
POST	Post-Condition
BR	Business Rule

Content

Name **Page #**

1) CHAPTER 01: PROJECT INTRODUCTION.....9

1.1 – Project Background….....9

1.2 – Problem Statement… 10

1.3 – Motivation And Need… 11

1.4 - Aims and Objectives….....11

1.4.1 Objectives….....11

1.4.2 Benefits….....11

1.4.3 Deliverable….....11

1.5 - Problem Solution For Proposed System.....12

1.6 –Project Scope.....12

2) CHAPTER 02: LITERATURE REVIEW… 13

2.1 – Review….....13

2.2 – Person-job fit… 13

2.2.1 – Feature fusion for person-job fit… 13

2.2.2 – Two-way selection model….....13

2.3 – Job Recommendation System….....14

2.3.1 – Two-Layer Attention Mechanism….....14

2.3.2 – Learning job title similarity… 14

2.4 – Resume Analysis and Parsing.....14

2.4.1 – Data-driven resume analysis….....14

2.4.2 – Resume Parsing with BERT….....14

2.4.3 – Resume Parsing using NLP….....14

2.4.4 – Resume Screening with Machine Learning… 15

2.5 – AI-based Interview and candidate Evaluation.....15

2.5.1 – AI-powered Interview System… 15

2.6 –Comparison of Reviewed papers 15

2.7 –Research gap comparison table.....16

2.8 –Summary.....17

3) CHAPTER 03: RESEARCH APPROACH 18

3.1 – Development/Research Methodology… 18

3.2 – Development Model 19

3.3 – Justification… 19

3.4 – Project Flow 20

4) CHAPTER 04: REQUIREMENT ANALYSIS… 23

4.1 – Functional Requirements…. 23

4.2 – Non-Functional Requirements.....25

4.3 – Use Case Diagram.....27

4.4 – Use Cases Description…. 27

4.4.1 –Primary use case: Employer …. 27

4.4.2 – Primary use case: Candidate… 28

4.4.3 – Automate / System use case…. 29

4.4.4 – General use case… 29

4.5 –ERD Diagram.....30

4.5.1 –User and Role Management … 30

4.5.2 – Recruitment Workflow…. 30

4.5.3 – Interview and Evaluation System… 31

5) DESIGN AND ARCHITECTURE 32

5.1 – Architectural Overview.....32

5.2 –Frontend Architecture 32

5.3 – Backend Architecture…. 33

5.4 – Machine learning service Architecture…. 33

6) CHAPTER 06: IMPLEMENTATION 34

6.1 – Implementation Overview…. 34

6.2 –Frontend Implementation.....34

6.3 – Backend Implementation…. 35

6.4 – Resume Machine learning service…. 35

6.5 –AI Interview Phase Implementation 36

6.6 – Explainability and Reporting…..... 36

7) CHAPTER 07: CONCLUSION AND FUTURE WORK 37

7.1 – System Limitations and challenges…..... 37

7.2 –Future work..... 37

7.3 – Conclusion…..... 38

8) CHAPTER 08: REFERENCES 39

Chapter 1: Project Introduction:

1.1 Project Background:

This report is about a web-based AI system called "Xenon," a system planned to help HR professionals and hiring managers in screening and recommending candidates utilizing AI and big data on their platforms. In today's digitalized age, ease of access to quality talent is still a major concern for companies, particularly when it comes to dealing with the high volume of applications and inherent human bias. Regular means of candidate selection often rely on several types of external help, which limits the efficiency of the hiring team.

The main objective of this project is to form a web application that lets HR professionals to input job descriptions and candidate resumes through a digital interface. By utilizing machine learning strategies, particularly Natural Language Processing (NLP), the application will process candidate data and deliver real-time feedback using match scores and detailed reports. This not only allows for quick affirmation of the selected candidates but also empowers a seamless and user-friendly experience.

The presentation of this system has the potential to fill the gap between conventional manual screening and advanced technology, engaging HR professionals to connect with their data without depending on manual review. The system aims to improve hiring quality, communication, and independence for the HR department. This project outlines the goals of the project, system design, research approach, and expected outcomes. Additionally, it looks at the project's scope, development needs, prospective market, and application effect. The project is to provide a useful and effective solution that will greatly increase the business community's recruiting efficiency through a methodical and iterative development approach.

1.2 Problem Statement:

Existing recruitment systems face several critical limitations:

- Heavy dependence on manual resume screening.
- Keyword-based filtering that ignores semantic context.
- Limited explainability of automated decisions.
- Poor scalability under high application volumes.
- Complex interfaces that reduce HR autonomy.

Challenges like these leads to longer hiring cycles, inconsistent candidate evaluation, and reduced trust in automated tools. So there is a need for a system that can intelligently match candidates automatically to roles while remaining transparent, scalable, and easy to us.

1.3 Motivation and need:

Even with all the new technology today, many tools still do not help people with **screening and selection** enough. Most ways of **filtering and ranking resumes** are hard to use and often need someone else to help. This makes it difficult for people with **high hiring volumes** to be independent. This project's goal is to create an easy tool that helps them use **AI screening** on their own. Here are the main reasons for making the **"Xenon"** system:

- **Objectivity and Bias Reduction:** Hiring professionals often need to review countless resumes manually, which introduces human bias. We solve this problem by providing an independent digital interface for **HR professionals** to interact with **candidate data** through an objective AI algorithm.
- **Making Things Easy to Use:** By providing accessible features like **dashboard metrics** and **automated reporting** for match scores, the web application user interface makes it easy for **HR professionals** to interact with digital technology.
- **Helping with Talent Quality:** Users that face **high-volume recruiting** need special automated tools to learn and interact with large datasets. The **"Xenon"** system will help these users practice **analyzing and selecting top talent**, which makes hiring easier for them.
- **Using Technology:** Cloud computing and AI are the most accessible digital technologies available in the world right now. We can use the accessible technology to assist **HR professionals** by machine learning. Our system recognizes **skills and context** to quickly and correctly **recommend candidates**, bridging the gap between manual screening and digital automation.
- **System for the User:** The system is developed by having **HR professionals** in mind. We want this system to play its role to provide a better digitally accessible application to **HR teams so** they can use it with ease.

By doing this, this project aims to assist **HR professionals** to interact with digital technology independently. By using smart digital technology and focusing on the users' needs, the **"Xenon"** application will help and aid them to **screen and rank candidates** with ease and give them more independence.

1.4 Aims and Objectives:

1.4.1 Objective:

- Develop an AI recommendation engine for candidate-job matching.
- Integrate AI-powered interviews for structured evaluation.
- Provide explainable recommendations to recruiters.

1.4.2 Benefits:

- **Saves Time:** The application will **screen and rank candidates** automatically by machine learning, which will help save time and effort.
- **More Accurate:** By training an AI model, it becomes more accurate in **recommending top talent**.
- **Easier to Use:** This application has accessibility features that make this app easier to use for **HR professionals**.
- **Saves Money:** By providing AI-powered screening digitally through a simple and accessible device, it reduces the cost of extra hardware-based assistive physical devices.

1.4.3 Deliverable:

- **Data Collection:** A set of **Job & Resume Data (JRD)** though which the application can learn and train to recognize.
- **Data Cleaning Process:** Steps to make the input **resume data** easier to read and detect by the system.
- **Trained Machine Learning Model:** A trained model to help **HR professionals** by **recommending candidates**.
- **Testing Results:** Supports that the system is accurate and works correctly.
- **Ready-to-Use System:** A system that can **rank candidates** in real-time and can be used in **talent acquisition**.
- **Monitoring and Updates:** A plan of monitoring and updates that keeps the system working correctly over time.

1.5 Solution for Proposed Module:

The system uses ML models to analyze resumes, match skills, and conduct AI interviews. Results are presented in a recruiter dashboard with fairness metrics.

Expected Outcomes:

- A **web application** that **recommends candidates** digitally and assists **HR professionals** to interact with digital technology, helping them **screen and rank candidates** using AI.
- A simple design with navigation through **dashboard metrics and detailed reports** that reads out the **match score** and helps in navigation.
- A machine learning model using **Deep Learning (DL)** to correctly and effectively **match candidates**.
- Easy-to-follow guides for users and developers.
- Sharing the research in journals and meetings.
- A free set of **job & resume data** for everyone to use.

1.6 Project Scope:

The scope of Xenon includes:

- Resume and job description processing (PDF/DOCX).
- Semantic matching using deep learning embeddings.
- Recruiter dashboard with ranking and reporting.
- Modular architecture supporting future extensions.

Chapter 2: Literature Review:

2.1 Review:

The recruitment and hiring process has been a major research focus in recent years due to the rapid growth of digital job platforms and the increasing numbers of job applications. Traditional recruitment systems only rely on keyword-based filtering, which fails to capture the semantic meaning of skills, experience, and job requirements. This limitation leads to poor candidate-job matching and biased hiring decisions.

Recent research emphasizes using Machine Learning (ML)**,** Deep Learning (DL)**,** and Natural Language Processing (NLP) techniques to improve recruitment accuracy and fairness. These approaches transform resumes and job descriptions into meaningful representations that allow intelligent comparison and ranking. This chapter reviews key research papers related to job recommendation, resume parsing, candidate-job matching, and AI-based interviews, forming the foundation of the proposed Smart Recruitment Bot.

2.2 Person-Job Fit

2.2.1 Feature Fusion for Person-Job Fit (Jiang et al., 2020):

In this paper, the authors focus on improving job matching by combining different types of information, such as candidate experience, skill sets, and job requirements. Their feature-fusion technique shows that using multiple sources of data together leads to more accurate person-job matching compared to using single features alone [1].

2.2.2 Two-Way Selection Model (Chen Yang et al., 2022)

This paper explains that hiring should be seen as a two-sided process. It is important to check not only whether the applicant suits the job but also whether the job suits the applicant. Their model evaluates both sides, which makes the matching process more balanced and effective [2].

2.3 Job Recommendation Systems:

2.3.1 Two-Layer Attention Mechanism (Yu Mao et al., 2024):

The authors propose a job recommendation model that uses a two-layer attention mechanism. This technique helps the system focus on the most important information inside resumes and job descriptions. As a result, the recommendations become more accurate and meaningful [3].

2.3.2 Learning Job Title Similarity (2022):

This work aims to solve the issue of confusing or inconsistent job titles. The researchers train a model that learns similarities between job titles using skill-based data. This helps the system understand job roles better, even when different companies use different titles [4].

2.4 Resume Analysis and Parsing:

2.4.1 Data-Driven Resume Analysis (2016):

This is an early study that uses NLP and basic machine learning to read and analyze resumes. Although the approach is simple, it clearly shows how automated text processing can help HR teams handle large numbers of resumes more easily [5].

2.4.2 Resume Parsing with BERT (2019):

This paper uses the BERT deep-learning model to extract structured information from resumes and match it with job descriptions. Because BERT understands context well, it performs significantly better than older rule-based parsing methods [6].

2.4.3 Resume Parsing Using NLP (2024):

This study uses simple NLP techniques to extract important details like skills, experience, and education from resumes. It demonstrates an easy and lightweight way to automate resume reading without heavy deep-learning models [7].

2.4.4 Resume Screening with Machine Learning:

In this work, general machine-learning algorithms are used to classify and filter resumes. Although the approach is not very advanced, it is still helpful for automating basic resume-screening tasks [8].

2.5 AI-Based Interview and Candidate Evaluation:

2.5.1 AI-Powered Interview Systems:

Recent research also introduces complete solutions that go beyond resume screening. Some systems use AI to analyze resumes, simulate HR interviews, and even test technical skills. These models aim to provide a more interactive and supportive experience for job applicants [9].

2.6 Comparison of Reviewed Papers:

Paper/Study	Resume Parsing	Job Recommendation	Person-Job Fit	AI Interview / Evaluation
Jiang et al., 2020 – Feature Fusion			✓	
Chen Yang et al., 2022 – Two-Way Selection			✓	
Yu Mao et al., 2024 – Two-Layer Attention Model		✓		
Job Title Similarity Learning, 2022		✓		
Resume Analysis (2016)	✓			
Resume Parsing with BERT (2019)	✓			
NLP-Based Resume Parser (2024)	✓			
ML-Based Resume Screening	✓			

2.7 Research Gap comparison table:

Paper/Research Gap	Cold-Start Problem	Bias in Data	Lack of Explainability	Resume Format Issues	Limited Personalization
Jiang et al., 2020 (Feature Fusion)	☑ FA	☐ NA	☐ NA	☐ NA	☐ NA
Chen Yang et al., 2022 (Two-Way Selection)	☐ NA	☐ NA	☐ NA	☐ NA	☑ PA
Yu Mao et al., 2024 (Attention Model)	☐ NA	☐ NA	☐ NA	☐ NA	☑ PA
Job Title Similarity (2022)	☐ NA	☐ NA	☐ NA	☐ NA	☐ NA
Resume Analysis using NLP (2016)	☐ NA	☐ NA	☐ NA	☑ PA	☐ NA
Resume Parsing with BERT (2019)	☐ NA	☐ NA	☐ NA	☑ FA	☐ NA
Resume Parser (2024)	☐ NA	☐ Na	☐ NA	☑ FA	☐ NA
Resume Screening using ML	☐ NA	☐ Na	☐ NA	☑ PA	☐ NA
AI-Powered Interview System (2024)	☐ Na	☐ Na	☐ NA	☐ NA	☑ FA
LLM-as-an-Interviewer (2024)	☐ NA	☐ NA	☑ PA	☐ NA	☑ FA

Abbreviations

FA = Fully Addressed

NA = Not Addressed

PA = Partially Addressed

2.8 Summary:

This chapter reviewed existing intelligent recruitment systems and relevant research papers. The literature confirms that AI and NLP significantly enhance hiring accuracy. However, most systems lack interview evaluation and end-to-end automation, which is addressed by the proposed Smart Recruitment Bot.

Chapter 3: Research Approach:

3.1 Research Methodology:

The development of the Xenon follows an applied research methodology, as the objective is not just to study the existing recruitment systems but also to design, implement, and evaluate a practical AI-based solution. Our research approach combines theoretical foundations of existing literature with hands-on system development and experimentation.

The methodology begins with a literature review to understand current trends in resume parsing, job recommendation systems, and AI-based interviews. Based on the insights gained from research papers, identifying system requirements and then refining them. A structured workflow was then designed to transform unstructured textual data (resumes and job descriptions) into meaningful representations using NLP techniques.

The proposed methodology consists of the following phases:

1. **Requirement Analysis:**
   Functional and non-functional requirements were identified by analyzing real-world recruitment processes and limitations of existing systems.
2. **Data Collection:**
   Resume data and job descriptions were collected from publicly available datasets and sample recruitment platforms to simulate real hiring scenarios.
3. **Data Engineering Pipeline:**
   A pipeline was designed to preprocess, clean, and transform textual data into a machine-readable format.
4. **Model Development:**
   NLP-based similarity matching techniques were applied to calculate job-resume compatibility scores.
5. **AI Interview Integration:**
   A text-based AI interview module was introduced to further help to evaluate shortlisted candidates.
6. **Evaluation and Testing:**
   The system will be tested using different scenarios to ensure accuracy, performance, and reliability.

This structured methodology ensures that the system is developed systematically while maintaining alignment with academic research and real-world applicability.

3.2 Development Model:

The **Agile Development Model** is selected for the implementation of this project. Agile development emphasizes iterative progress, flexibility, and continuous improvement, making it well-suited for AI-based systems where experimentation and refinement are essential.

Instead of developing the entire system in a single phase, the project is divided into multiple iterations (sprints), each focusing on a specific module such as resume parsing, job matching, AI interview, or reporting. At the end of each iteration, the module was tested and improved based on feedback and observed results.

The key characteristics of the Agile model applied in this project include:

- Incremental development of features
- Continuous testing and validation
- Early detection of errors
- Flexibility to modify algorithms and workflows

This model helped in improving system accuracy and allowed adjustments to the matching logic and interview evaluation mechanism without affecting the entire system.

3.3 Justification:

The selection of the Agile Development Model is justified due to the nature of the project. AI and NLP-based systems often require experimentation with different preprocessing techniques, similarity measures, and scoring mechanisms. Agile allows these components to be modified and optimized during development.

Additionally, recruitment systems must be user-friendly and adaptable. Agile supports continuous feedback, which helps improve usability and functionality. Compared to traditional models such as the Waterfall model, Agile reduces development risk and ensures better alignment with project goals.

From a research perspective, Agile also supports iterative validation of results, enabling the evaluation of intermediate outputs before final deployment.

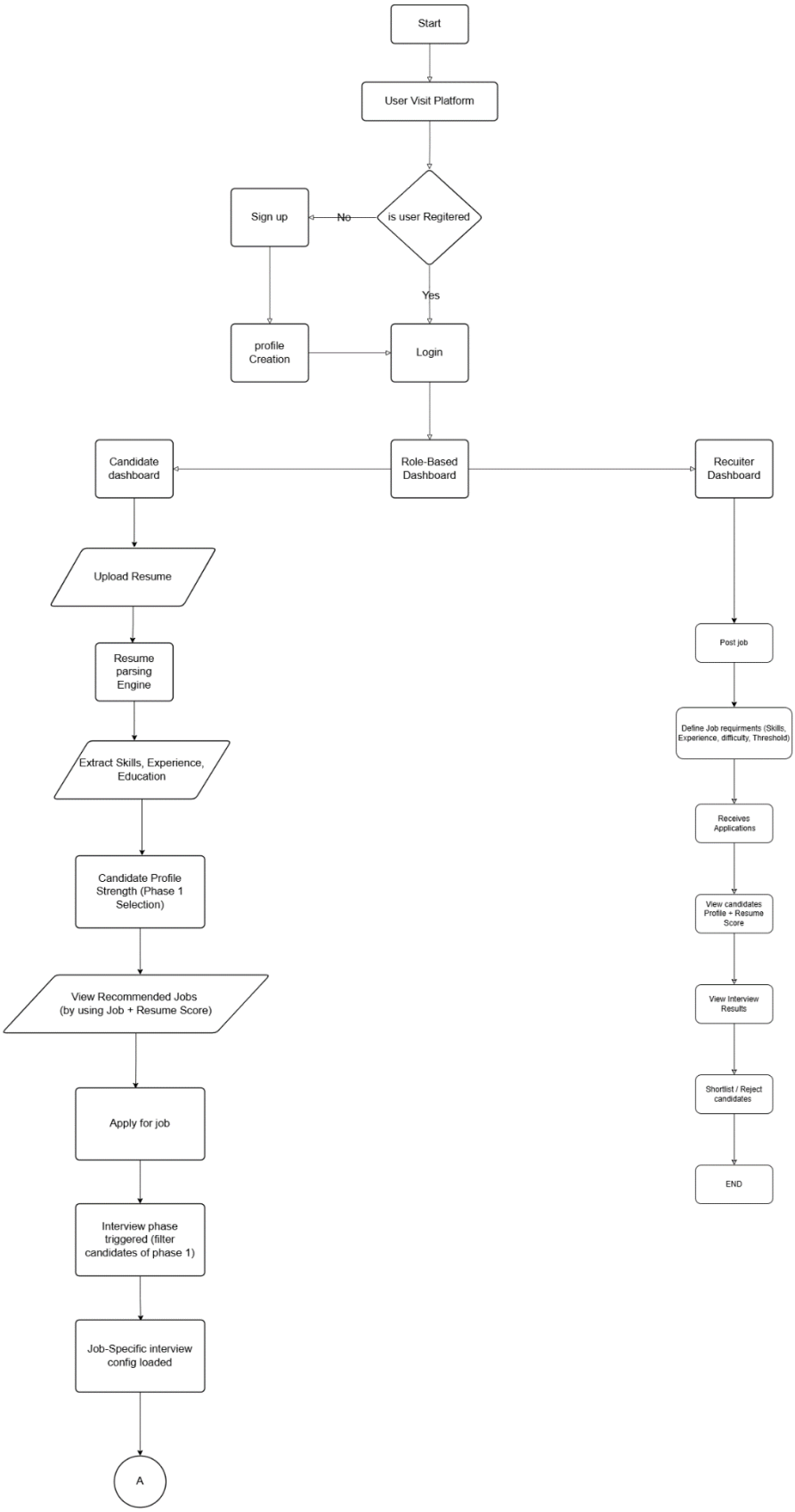
3.4 Project Flow:

The overall project flow of the Xenon is designed to automate the recruitment lifecycle from job posting to candidate ranking. The flow is as follows:

1. **Job Posting:**
   The recruiter posts a job description through the web interface.
2. **Resume Submission:**
   Candidates upload their resumes and apply for available jobs.
3. **Data Preprocessing:**
   Resumes and job descriptions are cleaned by removing stop words, punctuation, and irrelevant data.
4. **Feature Extraction:**
   Important skills, experience, and keywords are extracted using NLP techniques.
5. **Job-Resume Matching:**
   A similarity score is calculated between job requirements and candidate profiles.
6. **Shortlisting:**
   Candidates meeting the threshold score are shortlisted.
7. **AI Interview:**
   Shortlisted candidates participate in a text-based AI interview.
8. **Final Scoring:**
   Resume score and interview score are combined.
9. **Report Generation:**
   Recruiters receive ranked candidate reports and analytics.

This flow ensures a complete and automated recruitment process.

Project Flow diagram:


Start
User Visit Platform
is user Regitered
No
Sign up
Yes
profile Creation
Login
Role-Based Dashboard
Candidate dashboard
Recuiter Dashboard
Upload Resume
Resume parsing Engine
Extract Skills, Experience, Education
Candidate Profile Strength (Phase 1 Selection)
View Recommended Jobs (by using Job + Resume Score)
Apply for job
Interview phase triggered (filter candidates of phase 1)
Job-Specific interview config loaded
A
Post job
Define Job requirments (Skills, Experience, difficulty, Threshold)
Receives Applications
View candidates Profile + Resume Score
View Interview Results
Shortlist / Reject candidates
END

A
Adaptive Questions Generator (GPT)
Ask question
Candidate Answers
Answer Evaluation Engine
Score Vs job threshold
Ask next question difficulty (easy/ medium/ hard)
Interview completed
Final Interview Score + feedback
Send result to Recuiter
Notify by Email
Accept
Reject
END

Chapter 4: Requirement Analysis:

4.1 Functional Requirements:

Functional requirements tells what the system is supposed to do. For the Smart Recruitment System with Data Engineering Pipeline for Job Matching, the functional requirements were identified by analyzing real-world recruitment workflows and mapping them to automated system functionalities. These requirements ensure that both recruiters and candidates can effectively interact with the system.

The system supports two primary user roles: candidate and recruiter (HR professional).

FR-1: User Registration and Authentication

The system shall allow candidates and recruiters to create accounts and log in securely. Authentication ensures that only authorized users can access system features based on their role.

FR-2: Job Posting

The system shall allow recruiters to create and publish job descriptions by entering job title, required skills, experience level, and job details. These job descriptions serve as the main input for the job matching process.

FR-3: Resume Upload

The candidates are allowed to upload their resumes in supported formats (PDF/DOCX). Uploaded resumes will be stored temporarily for processing and analysis.

FR-4: Resume Parsing and Preprocessing

The system shall extract textual content from uploaded resumes and perform preprocessing tasks such as tokenization, stop-word removal, and normalization using NLP techniques.

FR-5: Job–Resume Matching

The system then compare processed resumes with job descriptions and calculate a matching score using similarity-based algorithms. This score identify how well a candidate fits a particular job.

FR-6: Candidate Shortlisting

The system shall automatically shortlist candidates whose matching score exceeds a predefined threshold set by the recruiter.

FR-7: AI-Based Interview

The system then conducts a text-based AI interview for shortlisted candidates. Questions are presented sequentially, and candidate responses are recorded for evaluation.

FR-8: Interview Evaluation

The system analyzes candidate responses using predefined rules and NLP-based scoring techniques to generate an interview score.

FR-9: Final Candidate Ranking

The system shall combine resume matching scores and interview scores to generate a final ranking of candidates.

FR-10: Report Generation

The system shall generate detailed reports for recruiters, including candidate rankings, individual scores, and evaluation summaries.

FR-11: Application Status Tracking

The system shall allow candidates to view the status of their job applications (e.g., applied, shortlisted, interviewed, and selected/rejected).

Table 4.1: Functional Requirements Summary

ID	Requirement Name	Description
FR-1	Authentication	Secure login for users
FR-2	Job Posting	Recruiters post job descriptions
FR-3	Resume Upload	Candidates upload resumes
FR-4	Resume Parsing	Extract and preprocess resume data
FR-5	Job Matching	Calculate job-resume match score
FR-6	Shortlisting	Automatic candidate filtering
FR-7	AI Interview	Conduct text-based interview
FR-8	Interview Scoring	Evaluate candidate responses
FR-9	Ranking	Generate final candidate ranking
FR-10	Reporting	Generate recruiter reports
FR-11	Status Tracking	Candidate application tracking

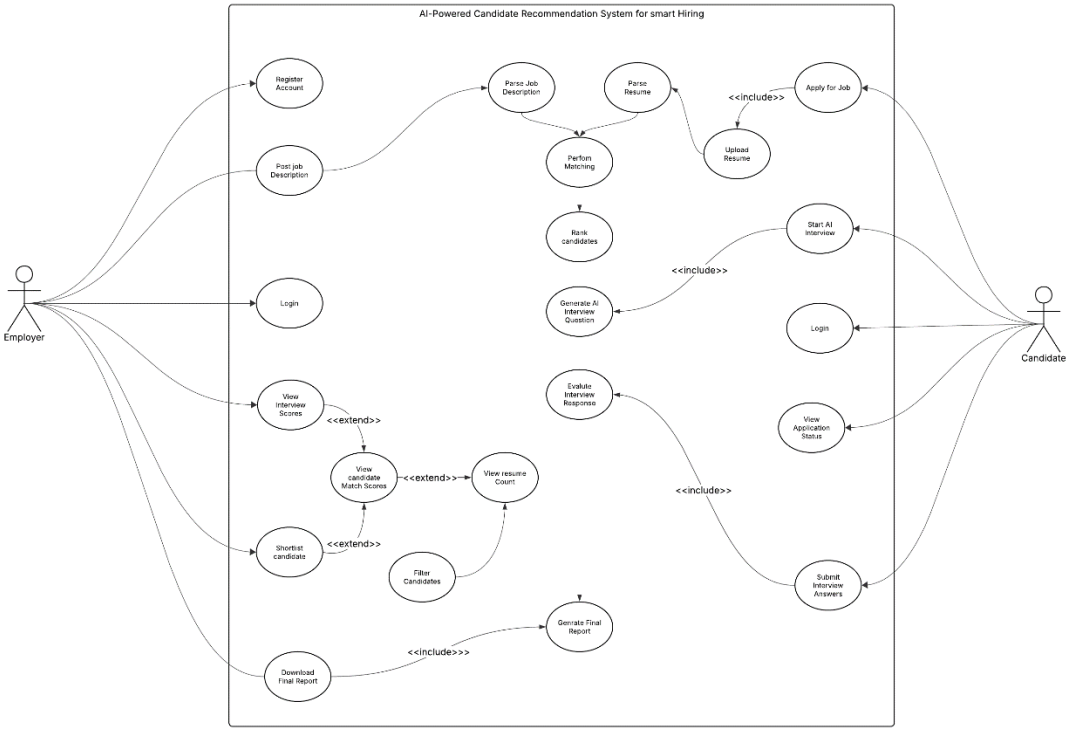
4.2 Non-functional Requirements:

Non-functional requirements define how well the system performs its functions**.** These requirements ensure system quality, reliability, and usability.

Identifier	**Title**	**Requirement**
Performance (PE)		
PE-1	**Matching Speed**	The system will rank a candidate's resume against a job description quickly, in less than **5 seconds**, so it feels fast and real-time.
PE-2	**System Responsiveness**	The system will respond quickly when users click or interact, with no lag or freezing. It will be smooth and easy to use.
PE-3	**Resource Efficiency**	The system will run well on different deployment servers (e.g., cloud), without using too much memory or processing power.
PE-4	**Scalability**	The system will handle more data or bigger tasks (e.g., a batch upload of 1000 resumes) without slowing down, even as it grows or gets more complicated.
Usability (US)		
US-1	**Learnability**	This application provides an easy-to-understand platform that the users using it for the first time can easily adapt to, specially the **HR professionals.**
US-2	**Ease of Use**	It provides clear steps and instruction for the **matching process** and provides accessible, easy-to-use navigation and input features.

US-3	**Interaction Efficiency**	It provides responsive user interaction and effective and accurate **candidate ranking** in real time.
Readability (RE)		
RE-1	**Code Quality**	The code must be well-structured, commented, and easy for other developers to understand and maintain.
RE-2	**Documentation**	The system must have comprehensive documentation for installation, maintenance, and usage.
Maintainability (MA)		
MA-1	**Modularity**	The system's architecture must be modular, allowing for independent updates of the DL model or the UI without breaking the whole system.
MA-2	**Upgradability**	The system must be designed to easily integrate future features, such as integration with Applicant Tracking Systems (ATS).

4.3 Use Case Diagram:



4.4 Use Case Diagram description:

4.4.1 Primary Use Case: Employer

UC1: Post Job Description

- **Actor:** Employer
- **Description:** The employer enters the requirements and details for a new job opening.
- **Flow:** The system receives the input and triggers the **Parse Job Description** internal process to extract key skills and requirements.
- **Outcome:** A job post is live and ready for candidate matching.

UC2: View Candidate Match Scores

- **Actor:** Employer
- **Description:** The employer reviews how well candidates align with the job requirements based on AI analysis.
- **Relationships:**

    - **Extends:** This extends the *View Interview Scores* and *Shortlist Candidate* use cases, providing additional data layers.
    - **Extended by:** View Resume *Count* can further extend this view to show volume data.
- **Outcome:** The employer makes data-driven decisions on which candidates to move forward.

UC3: Download Final Report

- **Actor:** Employer
- **Description:** The employer requests a comprehensive summary of a candidate's performance.
- **Relationships: Includes** generating the final report.

Outcome: A PDF or document is generated containing matching scores, interview evaluations, and AI recommendations.

4.4.2 Primary Use Case: Candidate:

UC4: Apply for Job

- **Actor:** Candidate
- **Description:** The candidate selects a job opening and submits their application.
- **Relationships: Includes** *Upload Resume*.
- **Flow:** Once the resume is uploaded, the system triggers **Parse Resume** and **Perform Matching** to compare the candidate against the job description.
- **Outcome:** The application is submitted, and a matching score is generated.

UC5: Start AI Interview

- **Actor:** Candidate
- **Description:** The candidate begins an automated screening interview.
- **Relationships: Includes** *Generate AI Interview Question*.
- **Outcome:** The system presents dynamic questions based on the job description and the candidate's profile.

UC6: Submit Interview Answers

- **Actor:** Candidate

- **Description:** The candidate records or types their responses to the AI questions.
- **Relationships: Includes** *Evaluating Interview Responses.*
- **Outcome:** The AI analyzes the sentiment, technical accuracy, and communication style of the answers.

4.4.3 Automated / System Use Case:

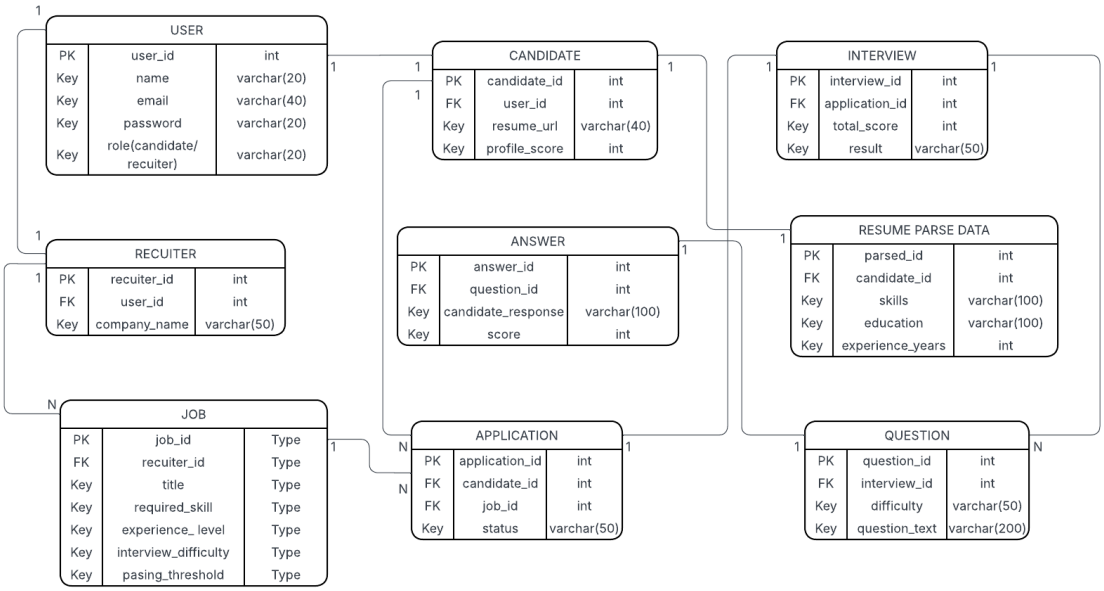
These are internal processes triggered by actor actions:

Use Case	Triggered By	Description
Perform Matching	Apply for Job	Compares parsed JD data with parsed resume data to create a compatibility score.
Rank Candidates	Perform Matching	Orders the pool of candidates from highest to lowest match for the Employer's view.
Evaluate Response	Submit Interview Answers	Uses NLP (Natural Language Processing) to score the quality of candidate answers.
Generate Final Report:	Download Final Report	Compiles all data points (resume match + interview score) into a final summary.

4.4.4 General Use Case:

- **Register/Login:** Standard authentication for both the employer and the candidate to access their respective dashboards.
- **View Application Status:** Allows the candidate to see if they are "Under Review," "Shortlisted," or "Rejected."

4.5 ERD Diagram:



This Entity Relationship Diagram (ERD) represents the database structure for the Xenon It defines how user data, job postings, applications, and AI-driven interview results are stored and related.

4.5.1 User and Role Management:

The system uses a centralized **USER** table to store basic credentials (email, password) and identifies whether the person is a **CANDIDATE** or a **RECRUITER.**

- **USER to CANDIDATE/RECRUITER:** This is a 1-to-1 relationship. Each user account is specialized into one of these two profiles.
- **RECRUITER:** Stores specific company information.
- **CANDIDATE:** Stores the resume link and an AI-calculated "profile score."

4.5.2 Recruitment Workflow:

This section handles the posting of jobs and the matching of candidates.

- **JOB:** Created by a recruiter (1-to-N). It defines the required skills and the "passing threshold" for AI evaluation.

- **APPLICATION:** This is a bridge table that connects a **CANDIDATE** to a **JOB**. It tracks the status of the hiring process (e.g., Pending, Accepted).
- **RESUME PARSE DATA:** Linked 1-to-1 with the Candidate. This table stores structured data (skills, education, and years of experience) extracted from the candidate's raw resume file by the AI.

4.5.3 Interview and Evaluation System:

Once an application is submitted, it triggers the automated interview process.

- **INTERVIEW:** Linked 1-to-1 with an Application. It records the total_score and the final result (Pass/Fail).
- **QUESTION:** Each interview contains multiple questions (1-to-N). These have varying difficulty levels and text.
- **ANSWER:** Stores the candidate's specific response to a question and the AI-generated score for that particular answer.

Chapter 5: Design and Architecture:

5.1 Architectural Overview:

Xenon is implemented using a scalable MERN-based architecture combined with a Python-based Machine Learning microservice. The system is designed to support modular development, independent scalability, and clear separation of concerns between presentation, business logic, and AI processing layers.

The overall architecture follows a **distributed three-layer model**:

- **Frontend Layer (React.js):** Responsible for user interaction, data visualization, and request orchestration.
- **Backend Layer (Node.js + Express.js):** Handles authentication, business logic, API orchestration, and data persistence.
- **AI/ML Layer (Python + FastAPI):** Performs resume parsing, NLP preprocessing, vectorization, and candidate–job matching.

This hybrid architecture allows the recruitment workflow to remain responsive while computationally intensive ML tasks are handled asynchronously.

5.2 Frontend Architecture:

The frontend is built using **React.js**, following a component-based architecture to ensure reusability and maintainability. State management is handled using **React Query** for server-state synchronization and **Zustand** for lightweight client-side state management.

Key Components:

- **Authentication Module:** Secure login and role-based access (HR/Admin).
- **Job Management Interface:** Create, edit, and submit job descriptions.
- **Resume Upload Module:** Drag-and-drop multi-file upload with validation.
- **Matching Dashboard:** Displays match scores, candidate ranking, and summary metrics.
- **Detailed Report View:** Shows explainable AI outputs, skill overlap, and justification.

The frontend communicates with the backend using **RESTful APIs** via Axios and ensures real-time UI updates without page reloads.

5.3 Backend Architecture:

The backend is implemented using **Node.js with Express.js**, following a **service-oriented architecture**. It acts as the central coordinator between the frontend and the ML service.

Core Backend Responsibilities:

- User authentication and authorization (JWT-based).
- Job and candidate data management.
- API gateway for ML service communication.
- Request validation, logging, and error handling.

Database Layer:

- **MongoDB** is used as the primary database.
- Collections include Users, Jobs, Candidates, MatchResults, and AuditLogs.
- Indexed fields are used for fast retrieval and analytics.

5.4 Machine Learning Service Architecture:

The AI layer is implemented as an independent **Python FastAPI microservice**, enabling language-specific optimization and independent scaling.

ML Service Responsibilities:

- Resume and job description parsing.
- NLP preprocessing and text normalization.
- Feature embedding and semantic similarity computation.
- Match score generation and explainability output.

The ML service exposes REST endpoints that are consumed by the Node.js backend.

Chapter 6: Implementation:

6.1 Implementation Overview:

The implementation of **Xenon** consists of two tightly integrated intelligent components:

1. **AI-based Candidate Recommendation System**, and
2. **AI-powered Interview System**.

The system is implemented using a MERN stack for the core web application, supported by a Python-based machine learning microservice for resume matching and interview analysis. This modular design ensures scalability, fault isolation, and efficient handling of AI workloads.

The implementation phase converts the proposed architecture into working software modules, including frontend interfaces, backend services, machine learning pipelines, and interview evaluation mechanisms.

6.2 Frontend Implementation (React.js):

The frontend is developed using **React.js** with a component-based architecture to support modular development and reusability.

Key Frontend Modules:

- **Authentication Module:** Secure login for HR professionals and candidates.
- **Job Management Module:** Creation and management of job descriptions.
- **Resume Upload Module:** Drag-and-drop upload with file validation.
- **Candidate Ranking Dashboard:** Displays match scores and ranked candidates.
- **AI Interview Interface:** Presents interview questions and captures candidate responses.
- **Interview Report View:** Displays interview performance and evaluation results.

State management is handled using **React Query** for server data synchronization and **Zustand** for UI-level state. RESTful communication with the backend is performed using **Axios**, ensuring robust error handling and response consistency.

6.3 Backend Implementation (Node.js and Express.js)

The backend is implemented using **Node.js with Express.js**, following a layered service-oriented architecture.

Core Backend Responsibilities:

- User authentication and authorization using **JWT**.
- Secure storage of job descriptions, resumes, and interview data.
- API orchestration between frontend, ML services, and interview modules.
- Interview session management and response storage.
- Aggregation of resume scores and interview results.

Database Layer:

MongoDB is used to store:

- User profiles
- Job postings
- Candidate records
- Resume match results
- Interview responses and scores
- Audit and activity logs

Indexes are applied to frequently accessed fields to optimize performance.

6.4 Resume Matching Machine Learning Service:

The resume screening functionality is implemented as a **Python FastAPI microservice**, dedicated to candidate–job matching.

Resume Matching Workflow:

1. Resume and job description text is received from the backend.
2. NLP preprocessing is applied (tokenization, lemmatization, and normalization).
3. Semantic embeddings are generated using transformer-based models (e.g., Sentence-BERT).
4. **Cosine similarity** is computed between jobs and resume embeddings.
5. A normalized match score (0–100%) is returned to the backend.

This semantic approach enables accurate candidate ranking beyond simple keyword matching.

6.5 AI Interview Phase Implementation:

After resume-based shortlisting, selected candidates proceed to the **AI Interview Phase**, which is a core component of Xenon.

Interview Workflow:

1. The system dynamically generates interview questions based on the job role.
2. Candidates respond through a structured interview interface (text-based).
3. Responses are sent to the ML service for evaluation.
4. NLP models analyze responses for:
    - Relevance
    - Technical correctness
    - Communication clarity
    - Behavioral indicators
5. An interview score is generated and stored.

The AI interview system ensures consistency across candidates and reduces interviewer bias by applying standardized evaluation criteria.

6.6 Explainability and Reporting:

For both resume screening and interviews, Xenon provides explainable outputs:

- Skill overlap and missing skill indicators.
- Resume match score justification.
- Interview performance breakdown (technical, behavioral, communication).
- Final combined recommendation score.

These reports enable HR professionals to make informed decisions with confidence.

Chapter 7: Project Introduction:

7.1 System Limitations and Challenges:

Even though Xenon is made to help HR professionals match candidates using AI, visual feedback, and reporting, there are still some areas that need improvement. Right now, the system only works with standard resume formats (PDF/DOCX), so it cannot yet process video resumes or multiple job requirements at the same time. Another challenge is that the DL model sometimes has trouble with highly niche or complex job roles, which can cause mistakes in the matching score. Also, while the app is already very useful, it does not yet support integration with all Applicant Tracking Systems (ATS), so users cannot connect it to their existing HR infrastructure for a more accessible experience. The user interface (UI) is still being improved, and we are working on making it even easier to navigate based on user feedback. Even with these challenges, we are always working on improving accuracy, adding new features, and making the application more accessible so that it can help even more people in the future.

Scalability is also a potential challenge. While the system performs well for moderate datasets, handling extremely large volumes of resumes and concurrent users may require advanced optimization techniques and distributed processing. Additionally, bias in data is a concern. If training data or predefined rules are biased, the system may unintentionally favor certain profiles. Continuous monitoring and ethical evaluation are required to ensure fairness and transparency.

7.2 Future works:

We have plans to make Xenon even better and easier to use. One of the first things we want to do is add support for more languages and global resume standards so people from different countries can use the application comfortably. Not only that, but we also plan to improve the DL model by training it with more domain-specific job data and diverse resumes, making the matching more accurate and reducing mistakes. For now, our application only recognizes single-job matching. For the future, we want to have a feature for skill gap analysis and candidate training recommendations. We want to integrate our application in day-to-day HR platforms and Applicant Tracking Systems (ATS). Also want to integrate it in performance review and internal mobility planning for secure and independent transactions. Finally, we will keep improving the application design and performance by the user’s feedback. Our goal is to provide a user-friendly, accessible, and

easy-to-use digital application for HR professionals. These updates ensure Xenon becomes more helpful, accessible, and user-friendly for everyone.

7.3 Conclusion:

This project successfully designed and implemented a Smart Recruitment Bot with a Data Engineering Pipeline for Job Matching, addressing major inefficiencies in traditional recruitment processes. By leveraging Natural Language Processing and automation, the system reduces manual screening efforts, improves shortlisting accuracy, and enhances decision-making for recruiters.

The inclusion of an AI-based interview module adds an additional evaluation layer, making the recruitment process more objective and data-driven. The system demonstrates how AI and data engineering techniques can be effectively applied to solve real-world problems in human resource management.

Overall, the project fulfills its stated objectives and contributes a practical, scalable, and intelligent solution for modern recruitment systems. With further enhancements, the proposed system has the potential to evolve into a fully autonomous and enterprise-level recruitment platform.

Chapter 8: References:

1. Learning Effective Representations for Person-Job Fit by Feature Fusion (2020) by Junshu Jiang et al. – https://arxiv.org/abs/2006.07017
   PDF version: https://arxiv.org/pdf/2006.07017

2. Modeling Two-Way Selection Preference for Person-Job Fit (2022) by Chen Yang et al. – https://arxiv.org/abs/2208.08612

3. A Job Recommendation Model Based on a Two-Layer Attention Mechanism (2024) by Yu Mao et al. – https://www.mdpi.com/2079-9292/13/3/485

4. Learning Job Titles Similarity from Noisy Skill Labels
   https://arxiv.org/pdf/2207.00494

5. Data-driven HR Resume Analysis Based on Natural Language Processing and Machine Learning http://arxiv.org/pdf/1606.05611

6. End-to-End Resume Parsing and Finding Candidates for a job Description using BERT
   https://arxiv.org/pdf/1910.03089

7. Resume Parser Using NLP https://ijarcce.com/wp-content/uploads/2024/09/IJARCCE.2024.13905.pdf

8. Resume Screening using Machine Learning (PDF form).

9. AI-Powered Interview Preparation System: Integrating Resume Analysis, HR Simulation, and Technical Skill Assessment
   https://www.journaljerr.com/index.php/JERR/article/view/1489/3223